

ZIPA-CLI

Batch Phonetic Decoding with ZIPA Zipformer Models

User Manual

ONNX & PyTorch backends · multi-format I/O · timestamped alignment ·
model comparison · web viewer

Generated for the `zipa-cli` package.

Contents

1 Introduction

`zipa-cli` is a command-line tool for running *greedy phonetic decoding* with the ZIPA family of zipformer phone-recognition models (ACL 2025). It wraps the model checkpoints in a single streaming pipeline so you can transcribe one file or millions without reloading the model, across many input formats, with either a minimal-dependency ONNX backend or the full PyTorch backend.

Capabilities at a glance.

- Inputs: single file, file list, directory, CommonVoice-style TSV, STM segments, HuggingFace dataset, lhotse/icefall manifests, and lhotse shar.
- Backends: **ONNX** (default, minimal deps) and **PyTorch** (icefall/k2, GPU).
- Outputs: `tsv`, `jsonl`, `lhotse manifest`, `recogs`, `stm`, `ctm`, and `align-json`.
- Timestamped alignment, a two-model comparison mode (`match/sub/ins/del`, PER, PFER), a model registry with HuggingFace auto-download, and a client-side web viewer for phoneticians.

2 Installation

2.1 Minimal (ONNX backend — recommended)

```
pip install -e ".[onnx]"
```

Pulls `onnxruntime` `soundfile` `librosa` `lhotse` `huggingface_hub` `sentencepiece`. No `torch/icefall/k2` required.

2.2 Full (PyTorch backend — large-scale GPU)

```
pip install -e ".[torch]"
# then install icefall + k2 matching your torch/cuda versions:
# https://icefall.readthedocs.io/en/latest/installation/index.html
```

2.3 Optional extras

```
pip install -e ".[hf]" # HuggingFace dataset input
pip install -e ".[analysis]" # panphon PFER in compare mode
pip install -e ".[all]" # everything
```

2.4 Configuration

	Variable / flag	Meaning	Default
<code>ZIPA_CLI_CACHE</code>		model download cache	<code>~/.cache/zipa-cli</code>
<code>ZIPA_REPO</code> / <code>-zipa-repo</code>		path to the ZIPA training repo	auto-detected
<code>-tokens</code>		ONNX <code>tokens.txt</code>	bundled <code>ipa_simplified/tokens.txt</code>
<code>-bpe-model</code>		PyTorch <code>sentencepiece</code> model	bundled <code>unigram_127.model</code>

3 Models

zipa-cli knows all 12 released averaged checkpoints by short tag.

```
zipa-cli models list
zipa-cli models info zipa-cr-l-500k
zipa-cli models download zipa-cr-s-300k --backend onnx --precision fp16
```

Tag pattern	Architecture	Example
zipa-t-{s,l}-{300k,500k}	transducer	zipa-t-l-500k
zipa-cr-{s,l}-{300k,500k}	CR-CTC (ctc)	zipa-cr-s-300k
zipa-cr-ns-{s,l}-{700k,800k}	CR-CTC Ns	zipa-cr-ns-l-800k
zipa-cr-ns-nd-{s,l}-...	CR-CTC Ns, no diacritics	zipa-cr-ns-nd-l-780k

Anywhere a model is required you may pass a **tag** (auto-downloaded and cached) or a **local path** (an .onnx file or transducer directory, or a .pth checkpoint).

4 Quick start

```
# Single file (prints phones)
zipa-cli transcribe sample.wav --model zipa-cr-s-300k

# A directory of audio on GPU, dynamic batching, to a TSV
zipa-cli decode --input audio_dir/ --input-type dir \
  --model zipa-cr-l-500k --backend onnx \
  --max-duration 600 -o transcripts.tsv

# Compare two models' outputs
zipa-cli decode --input audio_dir/ --model zipa-cr-s-300k -o small.tsv
zipa-cli decode --input audio_dir/ --model zipa-cr-l-500k -o large.tsv
zipa-cli compare --a small.tsv --b large.tsv
```

5 Input sources

Select with `-input-type`; auto (the default) infers from the path/flags.

	Type	What it is	Key flags
file	one audio file		—
list	text file of audio paths		—
dir	directory searched recursively		—
tsv	id/path (+ ref) columns, CommonVoice-style		-id-col -path-col -ref-col
stm	file spk chan start end [text] segments		-audio-dir
hf	a HuggingFace dataset		-hf-dataset -hf-split -hf-config
manifest	lhotse/icefall manifests		-cuts <i>or</i> -recordings -supervisions
shar	directory of lhotse shar shards		—

5.1 Examples

```
# CommonVoice TSV (segid -> clip)
zipa-cli decode --input cv/test.tsv --input-type tsv \
  --id-col path --path-col path --ref-col sentence \
  --model zipa-cr-l-500k -o cv.tsv

# STM segments + an audio directory
zipa-cli decode --input show.stm --input-type stm --audio-dir audio/ \
  --model zipa-cr-l-500k --output-format stm -o show.hyp.stm

# HuggingFace dataset (streaming)
zipa-cli decode --input-type hf --hf-dataset mozilla-foundation/common_voice_17_0 \
  --hf-config en --hf-split test --audio-column audio \
  --model zipa-cr-l-500k -o cv17_en.jsonl --output-format jsonl

# icefall manifest (recordings + supervisions)
zipa-cli decode --input-type manifest \
  --recordings data/recordings.jsonl.gz --supervisions data/supervisions.jsonl.gz \
  --model zipa-cr-l-500k -o hyp.manifest.jsonl --output-format manifest

# Precomputed fbank cuts (feature extraction skipped automatically)
zipa-cli decode --input-type manifest --cuts data/cuts.jsonl.gz \
  --model zipa-cr-l-500k -o hyp.tsv

# lhotse shar shards (cuts/*.jsonl.gz + recording/*.tar)
zipa-cli decode --input data-shar/ --input-type shar \
  --model zipa-cr-l-500k -o hyp.tsv
```

6 Output formats

	Format	Description
tsv (default)	id<TAB>p h o n e s	
jsonl		rich record per utterance (add <code>-timestamps</code> for an alignment field)
manifest		lhotse SupervisionSet; round-trips into k2/lhotse
recogs	hyp=/ref=	lines parsed by <code>zipa/scripts/evaluate.py</code>
stm		file spk chan start end phones
ctm		NIST CTM: recording channel start dur phone (timed)
align-json		one JSON line per utterance with timed phones; feeds the web viewer

When the input carries a reference (TSV `-ref-col`, STM text, manifest supervisions) it is preserved into `jsonl/recogs/align-json` for evaluation. `-skip-existing` resumes a run by appending only new ids.

7 Batching, device, and precision

- `-batch-size N` — fixed utterances per batch, *or*
- `-max-duration S` — dynamic, lhotse-style: cap pooled audio seconds per batch (utterances length-sorted within a buffer to minimise padding).
- `-num-workers N` — audio/feature loading workers.
- `-device auto|cpu|cuda|cuda:N` — PyTorch device; ONNX uses the CUDA execution provider automatically when available.
- `-precision fp32|fp16|int8` — ONNX precision to download/use.

8 Timestamps and alignment

ctm and align-json always emit per-phone start/end times; `-timestamps` additionally attaches an alignment array to jsonl.

```
zipa-cli decode --input utt.wav --model zipa-cr-l-500k \
  --output-format ctm -o utt.ctm

zipa-cli decode --input audio_dir/ --input-type dir --model zipa-cr-l-500k \
  --output-format align-json -o aligned.json
```

Caution. *Timestamped decoding currently requires the ONNX backend, and ONNX timings are approximate — see Section ??.*

9 Comparison / analysis mode

```
zipa-cli compare --a modelA.tsv --b modelB.tsv \
  [--format txt|jsonl|csv] [--top-k 20] [-o report.txt]
```

Treats **A** as reference and **B** as hypothesis, joins by id, and reports: corpus and per-utterance *Match / Substitution / Insertion / Deletion* counts, $\mathbf{PER} = (S + I + D)/N_{ref}$, **PFER** (panphon articulatory feature edit distance; install [analysis]), the most frequent substitution pairs and inserted/deleted phones, and the most-disagreeing utterances. Inputs may be tsv or jsonl transcripts from decode.

10 Web viewer for phoneticians

A zero-install, client-side viewer lives in web/. Produce align-json for each model, then serve the folder:

```
zipa-cli decode --input utt.wav --model zipa-cr-l-500k --output-format align-json -o
  large.json
zipa-cli decode --input utt.wav --model zipa-cr-s-300k --output-format align-json -o
  small.json
cd web && python -m http.server 8000 # browse http://localhost:8000
```

Load the audio file and both JSON files; entries sharing an utterance id are stacked as one phone tier per model, positioned by timecode, over a waveform/spectrogram view.

11 Caution: the ONNX length heuristic

The two backends compute the number of decodable output frames differently, *by design*.

Backend	Output-length source	Time stride / frame
ONNX CTC	feat_lens // 2, clipped to session output	≈ 0.02 s
ONNX transducer	feat_lens // 4, clipped	≈ 0.04 s
PyTorch (both)	exact encoder_out_lens	exact

Why //2 for CTC when subsampling_factor is 4? The CR-CTC head emits at roughly half the encoder stride, so its log-probs come out at $\sim 2\times$ the encoder frame rate. The heuristic (carried verbatim from inference/batch_inference.py) approximates only the *trailing* valid length; it does not change the bulk of the emitted tokens but can clip or extend the last few frames and it sets the time stride used to place phones on the timeline.

Recommendation. On your first real ONNX CTC run, spot-check the transcript and any `-timestamps` alignment against a known reference (or against the PyTorch backend, which is exact). For precise timings prefer `-backend torch`.

12 Faithfulness to the reference scripts

- Features: 80-dim lhotse kaldi fbank (`dither=0.0`, `snip_edges=False`), mono, 16 kHz — identical to `inference/utils.py`.
- Decoding is greedy everywhere (blank id 0; transducer context size 2, ≤ 3 symbols/frame).
- Tokenizers bound per backend: `tokens.txt` (ONNX) vs `unigram_127.model` (PyTorch).
- Architecture/size come from the registry, not from string-matching the path.

13 Troubleshooting

Symptom	Fix
<i>Could not find tokens.txt</i>	Pass <code>-tokens</code> or set <code>-zipa-repo/\$ZIPA_REPO</code> .
<i>No module named icefall</i>	PyTorch backend needs icefall+k2 installed separately.
<i>No transducer encoder/decoder/joiner ONNX files</i>	Wrong <code>-precision</code> or path; check <code>models info <tag></code> .
ONNX timings look off	Expected approximation; validate vs PyTorch (Section ??).
Web imports blocked	Serve over HTTP (<code>python -m http.server</code>); <code>file://</code> blocks ES modules.